Programming Assignment 5

Filename: `pa5.c`

You must include the following commented lines at the beginning of your code to declare authorship. Submissions that do not include these lines will receive a 10% deduction.

```
/* COP 3502C PA5
This program is written by: Your Full Name */
```

In this activity, you will practice implementing sorting algorithms in C, focusing on the following:

- Implementing standard binary search tree operations
- Maintaining subtree data during insertion and deletion
- Writing recursive functions that act as counters and checkers
- Applying insertion, deletion, and search using the given information
- Designing and supporting queries on binary search tree structures

Background

The Annual Cat Charm Tournament is hosting its competition. Each cat competes and receives a unique charm score from the judges. The organizing committee wants to maintain a binary search tree as the live leaderboard. Cats may register, get disqualified, and be evaluated through various queries to determine rankings and filter valid candidates throughout the competition.

You have designed this program to do the following:

- Efficiently insert and remove cats from the leaderboard
- Maintain subtree metadata
- Look up any cat's standing or determine its current rank
- Analyze breed diversity across score brackets
- Mass-disqualify cats based on trait conditions

Develop a program that manages the competition leaderboard using a binary search tree, supporting real-time updates and complex queries.

You will process a sequence of operations representing events during the competition. Cats register by being inserted into a binary search tree sorted by name. Cats may also be disqualified and removed at any point. During this process, the organizing committee will issue queries against the leaderboard to determine rankings and filter the stored data.

Variables and Context

The program takes input representing a sequence of operations for the competition leaderboard. Each operation either modifies the tree or queries it for information.

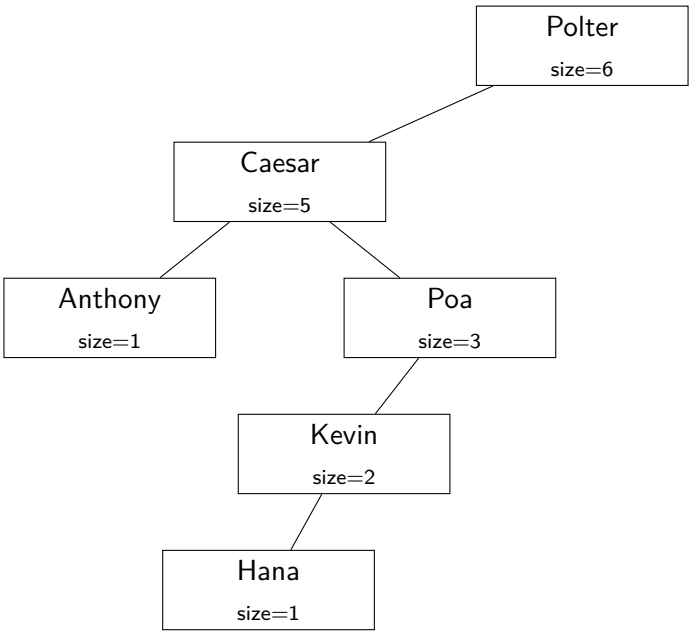
Each cat's information is stored in a `Cat` structure that contains a dynamically allocated name and breed, an integer charm score, and a statically allocated array of 5 trait flags. The name serves as the

tree search key. Each insertion and search should use `strcmp()` to navigate the tree. The charm score is stored as data within the node, but it does not determine the node's position in the tree.

Each trait flag corresponds to an index in the `TRAIT_NAMES` constant array: friendly [0], grumpy [1], playful [2], lazy [3], curious [4]. A value of 1 means the cat possesses that trait, while 0 means it does not.

Each node in the tree is stored as a `BST_Node` structure that contains a pointer to a `Cat`, pointers to the left and right child, and a `subtree_size`. The `subtree_size` represents the total number of nodes in that node's subtree (**not** the height).

For example:



Then Polter has `subtree_size = 6` because there are 6 nodes in its subtree, including itself. Caesar has `subtree_size = 5`, Anthony has `subtree_size = 1`, and so on. If a cat is deleted, every ancestor's `subtree_size` along the path must be decremented accordingly. In general, `subtree_size` can be computed as:

$$\texttt{subtree\_size} = \texttt{left subtree size} + \texttt{right subtree size} + 1$$

Important Note

The tree will not necessarily be balanced. Its shape depends entirely on the insertion order.

Required Data Structures

You must use the following structure in this assignment. You are allowed to declare more structures if needed. However, you are not allowed to change the required structure.

```c
typedef struct Cat_s {
    char *name;                 // dynamically allocated cat name
    char *breed;                // dynamically allocated cat breed
    int charm;                  // unique charm score
    int traits[NUM_TRAITS];     // values corresponding to TRAIT_NAMES
} Cat;

typedef struct BSTNode_s {
    Cat *cat;
    struct BSTNode_s *left;
    struct BSTNode_s *right;
    int subtree_size;           // size of this subtree, not height
} BSTNode;
```

Global Constant Variable

The following must be declared in your code:

```c
#define MAX_NAME 25     // max name of a cat
#define NUM_TRAITS 5    // total number of traits

const char TRAITS[NUM_TRAITS][MAX_LEN] = {
    "friendly", "grumpy", "playful", "lazy", "curious"
};
```

Constant Descriptions

- `NUM_TRAITS` – the total number of traits.
- `MAX_LEN` – the maximum length of all strings used in the program, including both cat names and trait names (including the null terminator).

Starter Code

Your task is to complete the given C program. You can access the starter code here.

Queries Explained

1. **Query 1: Insertion**
   Insert the given cat into the tree using the cat's name as the key. If a duplicate name is found, replace the existing node's data (`breed`, `charm`, and `traits`) only if the new cat has strictly more traits set to 1. Otherwise, ignore the insertion.
2. **Query 2: Deletion**
   Delete a cat by name using the three-case deletion approach taught in lecture. The cat may or may not exist in the tree. If the node has two children, you must use the successor (minimum node in the right subtree).
3. **Query 3: K-th Smallest Element**
   Given a value `k`, locate the k-th smallest node in the tree according to in-order traversal. You must use the stored `subtree_size` values to do this efficiently.
4. **Query 4: Filter by Trait**
   Given a trait index and trait value, create a dynamically allocated array of strings containing the names of all cats whose trait at that index matches the given value. The names must appear in alphabetical order. Refer to the implementation requirements for the required function.
5. **Query 5: Remove by Trait**
   Given a trait index and trait value, remove all cats whose trait at that index matches the given value. Output the number of nodes removed.
6. **Query 6: In-Order Traversal**
   Print all nodes in the tree using in-order traversal. For each node, print its name, charm score, and `subtree_size`.

Input and Output Specifications

The program reads input from a file named `tournament.txt`. The first line of input contains a single positive integer `n`, representing the number of operations. Each of the next `n` lines begins with a positive integer `q` where `1 <= q <= 6`.

> Due to how the auto-grader is set up, make sure there are no trailing spaces in your output. Terminate each output with a `\n` character.

When `q = 1`

Input

```
1 <name> <breed> <charm> <t0> <t1> <t2> <t3> <t4>
```

The remaining tokens are used to create a node and insert it into the BST keyed on the cat's name in alphabetical order. The `name` and `breed` are strings of at most 25 characters. The charm value is an integer. Each `t0, t1, ..., t4` is either 0 or 1.

If a cat with the same name already exists, replace the data only if the new cat has strictly more traits set to 1. Otherwise, ignore the insertion.

Output

After successfully inserting a new node, print:

```
Insert: <depth>
```

where depth is the distance from the root. The root has depth 0.

If an existing node was replaced, print:

```
Replaced
```

When `q = 2`

Input

```
2 <name>
```

The token after q is the cat name to delete. It is not guaranteed that this name exists in the tree.

Output

Regardless of whether a node was deleted, print:

```
Deletion Complete
```

When deleting a node with two children, you must use the successor (minimum node in the right subtree).

When `q = 3`

Input

```
3 <value>
```

The token after the query number is a positive integer `K`. Find the K-th smallest element in the tree based on alphabetical order.

If `K` is larger than the number of cats currently in the tree, print:

```
NO SMALLEST ELEMENT FOUND
```

Otherwise, print:

```
<name> <breed> <charm>
```

Example:

```
Input:  3 1
Output: Anthony Tabby 91
```

When `q = 4`

Input

```
4 <traitIndex> <traitValue>
```

The next two tokens are a trait index and a trait value (0 or 1). This query uses DMA (`malloc char**` to tree size), performs deep copies of names, and prints all cat names matching the given condition in alphabetical order.

Output

If no cats match the condition, print:

```
NONE FOUND
```

Otherwise, print the trait name followed by a colon and the matching cat names separated by spaces:

```
friendly: Coco Hana Kevin
```

When `q = 5`

Input

```
5 <traitIndex> <traitValue>
```

The next two tokens are a trait index (`0 <= traitIndex <= 4`) and a trait value (`0` or `1`).

Output

If no cats match the condition, print:

```
NONE REMOVED
```

Otherwise, print the count of removed cats as a single integer on its own line.

When `q = 6`

Input

```
6
```

There are no additional tokens.

Output

If the tree is empty, print:

```
EMPTY
```

Otherwise, print each node's name, charm score, and `subtree_size`, one node per line, in alphabetical order:

```
<name> <charm> <subtree_size>
```

Sample Input `tournament.txt`

```
20
1 Polter Ragdoll 85 1 0 1 0 1
1 Caesar Siamese 47 0 1 0 1 0
1 Poa Persian 72 1 1 0 0 1
1 Kevin Maine 33 0 0 1 1 0
1 Anthony Tabby 91 1 0 0 1 1
1 Hana DSH 68 0 0 1 0 0
6
3 1
3 4
4 0 1
4 3 1
2 Polter
6
5 0 1
6
2 Caesar
2 Hana
2 Kevin
6
3 1
```

Sample Output

```
Insert: 0
Insert: 1
Insert: 2
Insert: 3
Insert: 2
Insert: 4
Anthony 91 1
Caesar 47 5
Hana 68 1
Kevin 33 2
Poa 72 3
Polter 85 6
Anthony Tabby 91
Kevin Maine 33
friendly: Anthony Poa Polter
lazy: Anthony Caesar Kevin
Deletion Complete
Anthony 91 1
Caesar 47 5
Hana 68 1
Kevin 33 2
Poa 72 3
2
Caesar 47 3
Hana 68 1
Kevin 33 2
Deletion Complete
Deletion Complete
Deletion Complete
EMPTY
NO SMALLEST ELEMENT FOUND
```

Implementation Requirements / Run-Time Requirements

Please read this section carefully and make sure your implementation satisfies all of these requirements.

You are required to build a binary search tree where each node is of type `BSTNode`. You must only add a new node when the name does not already exist in the tree. If the name already exists, the replacement logic from Query 1 applies. You must only structurally delete a node when Query 2 or Query 5 is called.

> We will check this by looking at your code, not just by checking the output. It is possible to produce the correct output without following the required implementation, but a significant portion of the grade depends on following the implementation specifications.

Your deletion code must match the three-case approach taught in class. Implementing deletion in any other way will result in a 0 on this assignment.

If your program times out, it is most likely due to improper tree traversal.

Queries 1, 2, and 3: $O(h)$

Your code must run in `O(h)` time for insertion, deletion, and the k-th smallest query, where `h` is the height of the tree. You should begin at the root and move only along the path needed to find the target location. While doing so, you must correctly maintain `subtree_size`. Full-tree traversal is not allowed for these operations.

Although linear solutions may exist, the purpose of this assignment is to use the tree's logarithmic-style search behavior whenever possible.

Query 4: $O(n)$

This query requires a full traversal of the tree. You must implement the following function prototype exactly as specified:

```
char** filterByTrait(BSTNode *root, int traitIndex,
                     int traitValue, int *resultSize);
```

Initially, allocate a `char**` of size equal to the total number of nodes in the tree. Perform a deep copy for each matching node and allocate exactly enough space for each string. Before returning the array, use `realloc()` so that the returned array has no wasted space.

Query 5: $O(n \cdot h)$

This query consists of two distinct phases. First, traverse the tree to determine which nodes satisfy the condition. Second, delete those matching nodes. Ideally, these two phases should remain separate.

Query 6: $O(n)$

This query requires a full in-order traversal of the tree. For each node, print its name, charm score, and `subtree_size` on its own line. If the tree is empty, print `EMPTY`.

Hints and Recommended Approach

- Start by implementing insertion and search using `strcmp()` for comparisons.
- Verify that your tree is correctly ordered alphabetically before moving on to deletion and query logic.
- Test deletion thoroughly using all three deletion cases.
- Use Query 5 to help verify that values are being updated correctly, since some of the other queries depend on this information.

Additional Implementation Requirements

- Do not declare any global variables other than those explicitly specified in the assignment instructions.
- When reading strings, you must first store the input in a static `char` array. Then allocate the exact amount of memory required for the string and copy it using the `strcpy()` function, following the method demonstrated in class.
- All dynamically allocated memory must be properly freed in order to receive full credit.
- Your program must terminate with no memory leaks.
- Your code will be tested with large test cases and must run within the time limit set on Gradescope. Common reasons for timeouts include unnecessary loops, less efficient code, unnecessary traversal, and unnecessary sorting.

Clarifications

For questions or clarifications about the program requirements, please post them in the corresponding discussion thread on Webcourses. If additional clarification is needed, you are welcome to visit the TAs or the instructor during office hours. Requests for help with debugging should be brought to office hours rather than sent by email.

Tentative Rubric (Subject to Change)

A set of test cases will be used to evaluate whether your program produces the expected output. Each failed test case will result in a grade deduction per the rubric below. In addition to the sample test cases, we will use additional (hidden) test cases during grading. Therefore, passing the sample test cases does not guarantee full credit. You are strongly encouraged to thoroughly test your code against a variety of inputs. Because some test cases are hidden, your final score will not be visible until the submission has been fully graded.

Important: Any attempt to hard-code the expected output or otherwise circumvent the grading process constitutes a violation of academic integrity and will be handled in accordance with university policy.

If it is later discovered that a submission violates the course AI policy or any academic integrity rules, the assignment may be re-evaluated even after grades have already been posted on Webcourses. In such cases, previously awarded scores may be retracted and the submission may be regraded according to the course policies. Additional academic integrity actions may also be taken if warranted.

- If your code does not compile on Gradescope, you will receive a score of 0.
- If you modify, omit, or fail to use the required data structures, you may receive a score of 0.
- Failure to use dynamic memory allocation for storing data will result in a score of 0.
- Correct implementation of all functions and specifications: 30%.
- Properly freeing all dynamically allocated memory with zero memory leaks: 10%.
- Passing all test cases with exact output matching: 60%.